MyCodeSchool

Code review — py-siruri

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 2 — 2026-10-26 — sha 096287a

Ce am verificat: cele 10 reparații din runda 1 (`siruri.py` , `siruri2.py`), `siruri3.py` (fișa 3) și `main.py` . Fiecare funcție din fișa 3 am rulat-o pe **toate rândurile din tabel**; fiecare 🔴 de mai jos e reprodus prin rulare.

Pe scurt: runda 1 a mers foarte bine — **B1–B8 sunt toate închise**, un commit pe bug, exact cum trebuie. Din fișa 3, cinci funcții trec pe tot tabelul: `utilizator_din_email` , `suma_cifrelor_din_text` (cu tabelul de urmărire, foarte bine!), `caractere_diferite` , `numara_aparitii` (provocare!) și `suma_numerelor_din_text` (provocare, inclusiv numărul de la final, `"x5"` și `"2026"` — e cea mai grea din fișă). `valideaza_data` e o idee bună a ta, nu era cerută.

Ce rămâne: `siruri2.py` **nu mai poate fi importat** (o redenumire din M3 a stricat un rând), **3 funcții din fișa 3 dau alt rezultat decât tabelul**, iar **valoarea de pornire** a revenit — reparată în `cel_mai_mare_numar` , scrisă din nou greșit în `cel_mai_ieftin` .

Runda 1 — ce s-a închis (verificat prin rulare)

✅ B1 `prima_litera` · ✅ B2 `initiale` · ✅ B3 `este_palindrom` · ✅ B4 `ascunde_cardul` · ✅ B5 `prezentare` · ✅ B6 `cel_mai_mare_numar` (dar vezi B5 de mai jos) · ✅ B7 `analiza_note` · ✅ B8 `analiza_vanzari` · ✅ M1 · ✅ M2 · ⚠️ M3 redenumit în loc de mutat → B1 de mai jos · ⚠️ M4 tot un singur `print` · ✅ C4 · ⏳ C1, C2, C3, C5 neatinse (C3 de mai jos)

🔴 Critice

B1 🔴 `siruri2.py` crapă la import — `siruri2.py:472`

```
>>> import siruri2
TypeError: unsupported operand type(s) for %: 'list' and 'int'
```

La M3 ai redenumit `numere` în `numerele` , dar în condiție ai schimbat și variabila de buclă: `if numerele % 2 == 0` întreabă „e **toată lista** pară?", nu „e **numărul** par?".

De ce se vede abia acum: tot ce e scris în fișier în afara unui `def` (dicționarele de test, listele `[... for ...]`) se **execută** în clipa în care faci `import`. O greșeală acolo oprește importul, deci **nicio funcție** din `siruri2.py` nu mai poate fi folosită — nici cele corecte. N-ai văzut-o pentru că `main.py` nu mai importă din `siruri2`. Verificarea că fișierul „e scris corect" (sintaxa) trece; eroarea apare doar când rândul **rulează**.

💡 Cum se numește, în aceeași paranteză, variabila care ia pe rând fiecare număr? 💡 După orice redenumire: rulează un `main.py` care importă din fișierul atins.

B2 🔴 `data_iso` întoarce ziua în față — `siruri3.py:74`

```
data_iso("21.10.2026")  ->  '21-10-2026'      (tabelul: '2026-10-21')
data_iso("")            ->  'Data invalida'   (tabelul: '')
```

Ai pus zero în față corect (`05`, `03`), dar ordinea e tot cea de la intrare. Formatul ISO e **an-lună-zi** — tocmai de aceea se folosește: datele scrise așa se sortează corect ca text (`"2026-10-21" < "2027-03-05"`, pe când `"21-10-2026" > "05-03-2027"`).

Al doilea rând: fișa spune ce întoarce funcția pentru `""`, iar tu ai ales alt text. Când cerința fixează rezultatul unui caz, nu-l inventezi tu.

💡 Te uiți la tabel: care din cele trei bucăți vine prima la ieșire?

B3 🔴 `ziua_cea_mai_calda` numără zilele de la 0 — `siruri3.py:79`

```
ziua_cea_mai_calda("21,25,19")  ->  1     (tabelul: 2)
ziua_cea_mai_calda("9,2,5")     ->  0     (tabelul: 1)
ziua_cea_mai_calda("")          ->  ValueError: invalid literal for int(): ''
```

Toate 7 rânduri din tabel pică, dar logica e **corectă** — găsește ziua bună (inclusiv pe negative și la egalitate). Greșeala e una singură: `contor` pornește de la `0`, adică întorci **poziția** din listă (cum numără Python), nu **numărul zilei** (cum numără oamenii, prima zi = 1).

La `""`: `"".split(",")` nu dă o listă goală, ci `[""]` — o bucată goală. `int("")` nu are ce citi și crapă. Cazul ăsta trebuie oprit **înainte** de `split`.

💡 Ce valoare de pornire pentru `contor` (și pentru `pozitie`) face ca prima zi să fie 1? 💡 Cum ai tratat `""` în `cel_mai_mare_numar`?

B4 🔴 `cel_mai_ieftin` pornește de la 1000 — `siruri3.py:92`

```
cel_mai_ieftin("tv:1500,laptop:4200")  ->  ''      (corect: 'tv')
cel_mai_ieftin("")                     ->  IndexError: list index out of range
```

Exact B6 din runda 1, a treia oară: valoarea de pornire e una **care nu vine din date**. Pe tabelul fișei merge, pentru că toate prețurile sunt sub 1000; la primul produs mai scump de 1000, niciun preț nu e „mai mic decât 1000", deci numele rămâne `""`.

Regula (aceeași pentru maxim și minim): pornești de la **primul element** — atât valoarea, cât și numele. Atunci pornirea e sigur una din date și nu există niciun preț pe care să nu-l poată întrece. Ai scris-o corect în `ziua_cea_mai_calda`, cu 10 rânduri mai sus (`maxim = int(zile[0])`).

La `""`: `"".split(",")` = `[""]`, apoi `"".split(":")` = `[""]`, iar `produs[1]` nu există.

B5 🔴 runda 1 B6 — reparat în `cel_mai_mare_numar`, rămas în celelalte trei — `siruri2.py:282`, `:358`, `:389`

```
analiza_vanzari({"Retur": -100, "Cablu": -50})  ->  ('', 0, 0, 0)   (produsul maxim:
'Cablu')
```

Runda 1 spunea: „același start de la 0: `analiza_vanzari`, `analiza_vanzari1`, `analiza_prod`". Ai reparat **linia** semnalată, nu **regula**. Când găsești o greșeală, întreabă-te: unde am mai scris la fel? Caută `= 0` lângă `maxim` / `minim` în tot fișierul.

🟡 Importante

- **M1** Lipsesc două funcții din fișa 3: `telefon_valid` (5) și `comprimare` (9).
- **M2** `main.py` afișează tot o singură funcție (`numara_aparitii`), iar `caractere_diferite` e importat și nefolosit. Fișa cerea câte un `print` pe **fiecare rând** din tabel — cu ele ai fi văzut singur B2, B3 și B4 (pică pe tabel, nu pe un caz ascuns).
- **M3** Fișa 3 lucra fără liste construite de tine: `caractere_diferite` (`siruri3.py:104`) se poate face cu un text în loc de listă (`diferite = ""`, `diferite += caracter`) — `in` merge la fel pe text.

🟢 Mărunte

- **C1** `media` se calculează **în** buclă la fiecare produs (`siruri2.py:288–292`) — rezultatul e corect, dar ajunge o dată, după buclă.
- **C2** Indentare de 5 spații în `valideaza_data` (`siruri3.py:41–59`) și de 11 la `:98` — în Python mereu 4.

- **C3** Din runda 1, neatinse: `pass` după `return` (`siruri.py:13` , `:18`) · `== True` (`siruri2.py:169` , `:188` , `:206` , `:232`) · indentare de 3 și 5 spații (`siruri.py:29` , `:39`) · `cel_mai_scurt_cuvant("")` → `IndexError` (`siruri2.py:55`) · `ascunde_mijlocul("abc")` → `'abbc'` (`:22`).
- **C4** Rânduri goale la finalul `siruri3.py` ; `main.py` fără `def main()` (ca în modelul din fișa 3).

Before / After

#	Acum	Corect
B1	`[numar for numar in numerele if numerele % 2 == 0]`	`[numar for numar in numerele if numar % 2 == 0]`
B2	`return f"{zi}-{luna}-{anul}"`	`return f"{anul}-{luna}-{zi}"` (și `""` → `""`)
B3	`contor = 0` / `pozitie = 0`	numărătoarea pornește de la 1; `""` → `0` înainte de `split`
B4	`pret_minim = 1000` / `nume_produs_minim = ""`	ambele luate din **primul** produs; `""` → `""`
B5	`maxim = 0` (×3)	primul element, ca în `cel_mai_mare_numar`

Întrebări

1. Ce întoarce `"".split(",")` ? De ce nu e o listă goală, și ce înseamnă asta pentru orice funcție care face `split` și apoi `int` ?
2. De ce B1 nu apare când rulezi `main.py` acum, dar ar apărea dacă adaugi `from siruri2 import prezentare` ?
3. `cel_mai_mare_numar` (reparat) și `cel_mai_ieftin` (scris după) au aceeași problemă de pornire. Ce faci data viitoare, **înainte** să scrii prima linie a unui maxim/minim?

Runda 1 — 2026-10-21 — sha f627497

- ✅ B1 `prima_litera` felie în loc de poziție · ✅ B2 `initiale` fără `.upper()` · ✅ B3 `este_palindrom` spații + litere mari · ✅ B4 `ascunde_cardul` `[:-4]` vs `[-4:]` · ✅ B5 `prezentare` fără „ani”
- ✅ B6 `cel_mai_mare_numar` pornea de la 0 — 🔴 aceeași regulă rămasă în `analiza_vanzari/vanzari1/prod` (runda 2 B5)
- ✅ B7 `analiza_note` nume pornit de la `""` · ✅ B8 `analiza_vanzari` împărțire la 0
- ✅ M1 email fără cifră · ✅ M2 funcții duplicate · ⚠️ M3 variabile cu același nume (redenumite → runda 2 B1) · ⚠️ M4 `print` pe fiecare rând
- ✅ C4 · ⏳ C1 `pass`, C2 `== True`, C3 indentare, C5 `""` și texte scurte (toate în runda 2 C3)